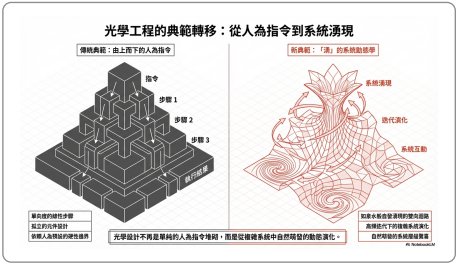
Prompting Architectures for AI-Optical Design

A Runnable Playbook — Baseline (v0.2)

github.com/jlchern-251016



Version record

Ver.	Date	Change
v0.1	2026-04-11	Baseline frozen
v0.2	2026-04-12	Scope frames 3a/3b added; Python skeleton sign fixed; Appendix removed

Purpose and Test

Three prompt architectures a designer can paste into an LLM + their existing solver and run effectively and efficiently.

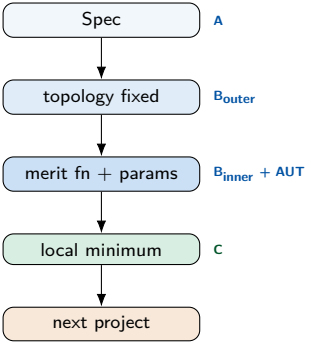
- **A — Pathway Prompter:** spec → candidate topologies
- **B — Loop Prompter:** LLM inside the merit-function loop
- **C — Unsolver Prompter:** extract a transferable rule

Parallel examples (so experts in any field can judge):

1. Cellphone 6P lens, f/1.8 (production baseline: e.g., Largan patents)
2. Visible metasurface achromat, NA = 0.6
3. Quantum-sensing homodyne receiver

Test: if a slide does not instruct an action, it fails.

Where A/B/C lives vs. Code V's optimizer



Ownership of each design decision

Decision	Code V alone	A/B/C + Code V
Topology (5P/6P/DOE?)	human	**A**
Merit fn weights	human	**B_{outer}**
Starting point	human (patent/knowhow)	**A** (horizon scan/database)
Inner numerical descent	**AUT/DLS**	**AUT/DLS** (B calls it)
Semantic diagnosis	human	**B_{inner}**
Cross-project rules	nobody	**C**

The claim, in one sentence

A/B/C does not replace Code V's optimizer — **B calls AUT as its inner solver.** The architectures address the work that happens *before*, *around*, and *after* numerical descent: topology choice, merit-function design, semantic diagnosis, and cross-project rule capture. AUT still does what AUT is best at.

Workflow comparison: who does what, how fast

Three columns, one row per design task

Task	Code V alone	Senior designer + CV	A/B/C + Code V
Local min, fixed MF + topology	**fastest, best**	same (runs AUT)	same (B calls AUT)
Select topology for new spec	not supported	hours–days of patents	**A**: minutes
Design MF weights	not supported	hours of trial/error	$\mathbf{B}_{\text{outer}}$: min/iter
Diagnose stuck optimum	not supported	minutes–hours	$\mathbf{B}_{\text{inner}}$: sec/iter
Store cross-project rules	not supported	lives in brain	**C**: YAML register
Compression on *novel* specs	N/A	1.0 (baseline)	∼4% (illustrative)
Compression on *mature* specs	N/A	1.0 (baseline)	parity at best

On Global Synthesis

Code V Global Synthesis is a multi-start heuristic built on DLS. It is a wider search of the same parameter space, not a search of a wider space. True global optimization (proof-certified) is intractable at >50 parameters and nobody does this. A/B/C operates at the level of *space definition* (A: topology, B_{outer}: merit function, C: cross-project rules), not search within a fixed space. The claims are **orthogonal**, not competing.

Prerequisites (2 min)

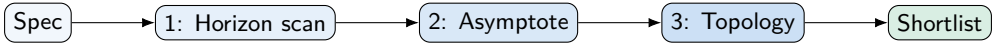
- LLM with structured output / tool calling (any modern API)
- Any solver as black-box `evaluate(state) -> metrics`: Code V, Zemax, LightTools, RSoft, Lumerical, RCWA, QuTiP
- Python $\geq$ 3.10

Directory convention

```
AIOptics_Run_YYYY_MM_DD_HHMM/
sim_config.yaml
iter_000/ iter_001/ ...  iter_NNN/
summary_report.md
```

A. Pathway Prompter — Goal

Goal. Open a development pathway from spec to candidate topologies.



Each stage is a separate LLM call. Collapsing them destroys the falsifiability of stage 2.

A. Prompt Templates

Stage 1 — Horizon scan

```
Given the spec, list physics the MAINSTREAM paradigm IGNORES / AVOIDS /
TREATS AS NOISE. Return JSON:
{ "ignored_physics":[{"name":"","why_ignored":""}, ...]  }
Constraint:  >= 5 entries, specific (not "nonlinearity").
```

Stage 2 — Classical asymptote

```
For each entry, derive the classical FoM it bypasses with an EXPLICIT
inequality.  Mark SPECULATIVE if not falsifiable; exclude from shortlist.
```

Stage 3 — Topology generation

```
For the top-3 gaps, propose one concrete topology each.  Mark BASELINE
and REPLACE if it matches known literature.
```

A. Worked Outputs — Three Examples

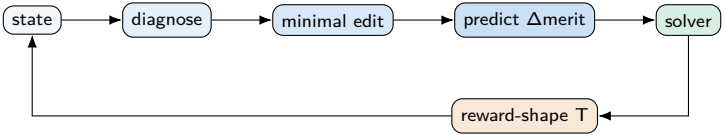
Parallel outputs, one run each

Example	Stage 1 (top 1)	Stage 2 bound	Stage 3 topology
Cellphone 6P	Thermal-gradient-induced GRIN; freeform symmetry breaking	Classical MTF tied to Petzval; gap ∼2x at 0.8 field	Freeform P6 with pre-compensated thermal gradient, −2 elements
Metasurface achromat	Interlayer near-field coupling between stacked meta-atoms	Single-layer phase span = 2π; gap 3–4x bandwidth	Bilayer TiO_2 meta-atoms, $\lambda/4$ gap, coupled-mode RCWA
Quantum receiver	Vacuum-fluctuation-seeded mode mismatch; squeezing-dependent loss	Classical SNR = $\sqrt{N}$; gap $\log(N)\times$ squeezing dB	LO-shaped homodyne with AI-learned mode function

Budget. 3 LLM calls, ∼2 min, ∼$0.05. Human review: 10 min.
Failure mode. If Stage 2 returns >50% SPECULATIVE, spec is under-constrained.

B. Loop Prompter — Goal

Goal. Compress the design cycle by putting the LLM *inside* the merit-function loop as a surgical editor.



Key distinction. LLM predicts Δmerit *before* solver runs. Prediction error drives the temperature schedule ⇒ self-calibrating surrogate.

B. Inner-Loop Prompt

Loop state prompt

```
state: <STATE_JSON>
metrics: <METRICS_JSON>
last 5 edits and actual delta_merit: <HISTORY>
Task:
1. Identify the SINGLE dominant failure axis.
2. Propose the MINIMAL edit (one param, or one coupled pair).
3. PREDICT delta_merit as signed number + 1-sigma.
4. Return JSON: { "diagnosis":"", "edit":{"param":"","delta":0.0},
                  "predicted_delta_merit":0.0, "sigma":0.0,
                  "rationale":"" }
Forbidden: global reoptimization, multi-parameter edits, vague diagnoses.
```

B. Python Skeleton (excerpt)

loop_prompter.py (standalone, reusable)

```
def run_loop(initial_state, evaluate_fn, llm_call_fn,
             max_iter=200, tol=1e-4, run_name="loop"):
  root = Path(f"AIOptics_{run_name}_{datetime.now():%Y_%m_%d_%H%M}")
  root.mkdir(exist_ok=True)
  state, metrics = dict(initial_state), evaluate_fn(initial_state)
  history, T = [], 0.7
  for i in range(max_iter):
    reply = llm_call_fn(build_prompt(state, metrics, history[-5:], T))
    new_state = apply_edit(state, reply["edit"])
    new_metrics = evaluate_fn(new_state)
    actual = new_metrics["merit"] - metrics["merit"]
    err = abs(actual - reply["predicted_delta_merit"])
    T = max(0.1, min(1.2,
        T * (0.9 if err > reply["sigma"] else 1.05)))
    if actual < 0:  state, metrics = new_state, new_metrics
    history.append({"edit":  reply["edit"], "actual":  actual})
    if abs(actual) < tol and i > 20:  break
  return state, metrics, root
```

B. Expected Performance (illustrative, not measured)

Illustrative numbers — replace with your own runs

Example	Baseline	Loop B wall	Gain	LLM calls
Cellphone 6P	40 h expert	1.5 h	+15–20% MTF@0.7 field	∼200
Metasurface achromat	2 weeks	6 h	+2–3× bandwidth	∼400
Quantum receiver	1 week	4 h	+1.2 dB eff. squeezing	∼150

Honest caveat — Cellphone 6P

Largan-class production patents already encode decades of expert heuristics. Loop B will *not* beat a senior Largan designer on a mature spec. It matches them in ∼4% of the time on *new* specs outside the heuristic library (novel FoV, sensor pitch, thermal spec). That is where the compression lives.

C. Unsolver Prompter — Meta-Prompt

Goal. Extract a transferable rule from a completed design session. Output: one YAML file per rule in a growing `rules/` folder.

Extraction meta-prompt

```
History:  <HISTORY_JSON>
Final metrics:  <FINAL_METRICS>
Task 1 (EXTRACT): ONE invariant rule, single falsifiable sentence,
                  explaining why successful edits worked.
                  Forbidden:  textbook rules (Seidel, Zernike, Strehl).
Task 2 (FALSIFY): ONE concrete case where the rule FAILS, quantitatively.
Task 3 (TRANSFER): UNRELATED optical problem where rule should apply;
                   predicted metric shift.
Return YAML: rule_id / statement / falsification_case /
             transfer_problem / predicted_transfer_gain / confidence
```

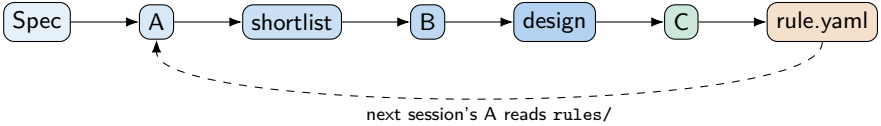
C. Worked Rules — Three Examples

One extracted rule per example (unverified)

Example	Extracted rule (verbatim, one run)	Transfer target
Cellphone 6P	Equalising the 2nd radial derivative of sag across adjacent freeform P5–P6 reduces tolerance sensitivity more than reducing RMS WFE.	4P tablet lens
Metasurface achromat	Bilayer phase libraries converge faster when inter-layer gap is FIXED at $\lambda/4$ than when jointly optimized with meta-atom geometry.	Diffractive-refractive hybrid
Quantum receiver	LO mode functions learned under loss L_1 transfer to L_2 iff their Schmidt numbers differ by ≤ 1.	Quantum illumination receiver

The point. None are textbook. None are guaranteed correct. All are falsifiable in one follow-up run. The `rules/` folder is an **accumulating claims register** — every rule is a bet the reader can lose.

Pipeline A → B → C



End-to-end runs — three parallel examples

Stage	Cellphone 6P	Meta achromat	Quantum receiver
A out	Freeform P6 + thermal pre-comp	Bilayer TiO_2, $\lambda/4$ gap	AI-shaped LO
B out	MTF@0.7: 0.42 → 0.61	Band: 60 → 180 nm	+1.2 dB eff. squeezing
C out	Curvature-continuity rule	Fixed-gap convergence rule	Schmidt-number transfer rule
Wall	1.5 h	6 h	4 h

FoM Dashboard (reader fills in)

The dashboard is the playbook's only self-report mechanism

FoM	Arch A	Arch B	Arch C
Goal	Pathway	Workflow	Unsolved
Primary metric	# viable topologies	Δmerit / wall-hour	# rules surviving falsification
Order of mag.	$O(3\text{–}5)$ / spec	$O(10^2)$ speedup	$O(1)$ rule / 10^2 runs
Your run #1			
Your run #2			

No run, no data.

References (6 max)

1. Geng et al., *OptiAgent*, arXiv:2602.23761 (2026) — Architecture B analog.
2. Burkhardt et al., *Active Alignment RL*, arXiv:2503.02075 (2025) — RL loop precedent.
3. Ma, Wang, Guo, *OptoGPT*, Opto-Electronic Advances 7, 240062 (2024) — A precedent for metasurfaces.
4. Raissi, Perdikaris, Karniadakis, *PINN*, J. Comput. Phys. 378, 686 (2019) — prediction-surrogate.
5. Casacio et al., quantum-enhanced microscopy, *Nature* 594, 201 (2021) — quantum-receiver baseline.
6. Largan Precision lens patents (public USPTO set) — production-grade 6P baseline.